

Centralized Training and Decentralized Control through the Actor-Critic Paradigm for Highly Optimized Multicores

Benedikt Dietrich, Heba Khdr, Jörg Henkel

Chair for Embedded Systems, Karlsruhe Institute of Technology (KIT), Karlsruhe, Germany
benedikt.dietrich@kit.edu, heba.khdr@kit.edu, henkel@kit.edu

Abstract—While distributed, neural-network-based resource controllers represent the state of the art for their ability to cope with the ever-expanding decision space, such approaches suffer from several limitations, like conflicting control decisions and partial observability. These effects can significantly impair the controllers’ learning capabilities and the stability of their control policies, causing substantial performance losses. We are the first to solve this problem employing a centralized training and decentralized control regime to mitigate the aforementioned limitations. Specifically, we design a centralized neural network (critic) that evaluates the behavior of multiple decentralized neural controllers (actors) in a system-wide context. The objective of our proposed technique is to maximize the performance under a temperature constraint through dynamic voltage frequency scaling. The evaluation of our technique shows its superiority over the state of the art, yielding average (peak) performance improvements of 20% (34%), which we consider a breakthrough as the gains are measured on a real-world platform.

I. INTRODUCTION

Runtime performance optimization is essential to get the most out of modern processors while adhering to system-imposed resource constraints, such as power budgets or temperature limits. Merely relying on design-time knowledge is increasingly insufficient for deriving efficient resource control policies that perform well in every possible runtime situation. Therefore, research shifted focus to online learning techniques. Reinforcement learning is particularly suited for adaptive resource control due to its two key properties [1].

- 1) It has the capability to find complex control strategies when it is challenging to define them analytically, provided only with the objective formulated as a reward signal. By framing resource control as a reinforcement learning problem, the underlying processor can be effectively treated as a black box, obviating the requirement for exhaustive design-time knowledge about the processor and the running applications.
- 2) It allows for continual adaptation to changing environment conditions by constantly monitoring the environment’s feedback to its decisions and accordingly updating the parameters of the control policy. This property is especially important in an open system, where workload combinations are virtually limitless, requiring frequent adjustments to the resource allocation between the running applications to maintain optimal performance.

Reinforcement learning has been proposed to implement centralized controllers, focusing primarily on coarse-grained control levers [2], [3], such as power capping or system-wide dynamic voltage and frequency scaling (DVFS), or operating within small solution spaces involving few cores and control decisions [4], [5]. However, centralized control is impractical for systems with multiple concurrently running applications and different workload characteristics, as the solution space grows exponentially with the number of cores and control knobs, slowing down convergence and incurring unacceptable runtime overhead. To enable scalable, finer-grained control, decision-making can be distributed among multiple lower-level controllers each focusing on a sub-system of the processor (e.g., a core or a cluster of cores). Distributed control introduces its own set of challenges [6]. With multiple controllers acting concurrently, it becomes increasingly difficult for each of them to trace the

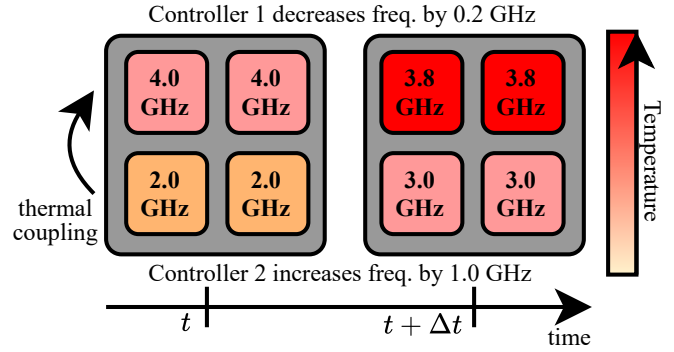


Fig. 1. Example to illustrate potential interference between controllers. Both controllers are assumed to operate independent of each other. They can only observe the cores under their supervision. However, the control problems are not independent and concurrent influence on the shared environment makes it challenging for each controller to trace the impact of its own actions. In this concrete example, the situation may arise that controller 1 observes an increase in temperature even if it has decreased its frequency due to a simultaneous frequency increase by controller 2 and thermal coupling between the cores.

impact of its own decisions on the processor’s state and system-wide objectives. This is known as the *credit assignment* problem in the reinforcement learning literature. Furthermore, each local controller has only a partial view of the entire system. Although this reduces the complexity of the learning problem, acting under *partial observability* also means that the controller can lack the complete information necessary to identify the optimal decision for the current situation. An additional challenge compared to the centralized setting is that the environment becomes increasingly *non-stationary*, as the same action can lead to different outcomes at different times as other controllers adjust their policies concurrently.

The following example shows these challenges in the context of processor resource control. We focus on the task of DVFS for temperature control. One controller is assigned to each running application and sets the frequency for the associated cores. Figure 1 illustrates how these changes affect an example processor with four cores. The initial frequency assignments are shown on the left, with controller 1 running its application at 4 GHz and controller 2 at 2 GHz. Since higher frequencies lead to higher core temperatures, controller 1 may decide to reduce its frequency by 0.2 GHz. Controller 2 measures that its current core temperature justifies an increase in frequency. After applying the new frequencies, the controllers measure the resulting temperatures in the next step. Since the control problems are interdependent due to thermal coupling among processor cores, the situation may arise where, despite controller 1 *decreasing* its frequency, it observes an *increase* in the temperature of its own cores.

Such counterfactual observations are typical for uncoordinated control, resulting in controller interference [7] and sub-optimal and/or unfair resource distributions [8]. To address these issues, prior research has explored hierarchical and self-organizing coordination schemes for distributed resource control [9], [10]. The approach in [9] is representative of a hierarchical technique in which a central arbiter configures the cost function of each local controller. It focuses on power-constrained performance optimization using per-core DVFS and distributes the system's power budget proportional to each core's utilization. However, this heuristic is insufficient for other, more complex objectives like maximizing system performance while adhering to a temperature constraint (due to spatial and temporal thermal effects). In [10], the goal is to optimize performance under a temperature constraint in a many-core processor. It uses one neural controller per core that tries to maximize the core's performance while keeping its temperature below the threshold. The state of each local controller is extended to include the temperature of its neighboring cores. Although this approach broadens the view of each controller, it does not fully resolve the challenges outlined above, as the controllers pursue individual performance objectives.

This paper is the first to approach resource control through *centralized training and decentralized execution* [6]. This paradigm allows us to address the inherent challenges of distributed resource control while still retaining the benefit of parallelizable decision-making and simplified learning problems of each controller. Specifically, we build on the *actor-critic paradigm* that distributes decision-making among multiple neural controllers, known as *actors*, and learns to evaluate the joint resource control using a second network, the *critic*. During centralized training, individual actors (controllers) share their local experiences with the critic, enabling it to evaluate each controller's decision-making in a global context. As a result, the actors can update their policies based on the critic's evaluation, aligning their decision-making with the *shared objective*: optimizing the overall system performance within the processor's limited resources. We apply this paradigm to the task of temperature control in a multicore processor within an open-system context, where various unknown applications may arrive to execute concurrently on the processor, competing for the thermal budget.

In summary, this work offers the following **novel contributions**.

- We propose a centralized training and decentralized execution approach for multicore resource control.
- Following this approach, we introduce an actor-critic-based technique to maximize the overall system performance under a temperature constraint through DVFS, while continuously adapting to varying workloads.
- We provide a lightweight implementation of our temperature control and assess its performance on a real processor, showcasing its ability to improve the performance within an open-system context, where a priori unknown applications are executing concurrently and competing for shared resources.
- We measure superior results compared to the state of the art on this real-world platform.

II. RELATED WORK

We give an overview of related work on (distributed) resource control. In addition, we provide relevant background information on cooperative reinforcement learning.

A. Multicore Resource Control

Multicore resource control has been a long-standing area of research. State-of-the-art resource control uses machine learning to enable application-aware decision-making, even for a priori unknown applications. For example, in a recent DVFS technique, the authors

of [11] use a long short-term memory network to classify application phases into known workload categories and apply the optimal frequency for each category. Although models trained at design time can generalize to unseen applications, it is virtually impossible to develop one that performs well across all potential workloads, environmental conditions, and device configurations likely to occur during operation. Reinforcement learning provides a promising alternative, enabling end-to-end learning of control decisions based on current performance counter and sensor measurements and adaptation to the aforementioned runtime dynamics. Recent works leverage deep reinforcement learning techniques, including value-based methods [10], [12] and policy-gradient methods [13]–[15] to implement control policies. In parallel, both centralized approaches with a single controller and decentralized approaches with multiple controllers have been proposed in the literature. Centralized approaches tend to focus on coarse-grained control levers [2], [3] or small solution spaces [4], [5]. To enable more fine-grained control, decision-making can be distributed among multiple lower-level controllers, each dedicated to a subsystem of the processor. Uncoordinated decision-making by these local controllers can result in suboptimal resource allocation and even destructive interference [7], [8]. To address these challenges, prior work has explored both self-organizing [10], [16] and hierarchical coordination schemes [9], [17]. Self-organizing systems rely on direct communication among agents, while hierarchical schemes decompose the global objective into local performance targets and resource constraints for each low-level controller. Some hierarchical approaches propose centralized searches in the joint action space of controllers [7], [18], [19]. However, such techniques require calling a central arbiter in every control interval, adding an additional step to the latency-critical control loop. Alternatively, custom resource distribution schemes have been developed for specific tasks, such as power-constrained performance optimization [9], [20]. These task-specific schemes are not easily transferable to other objectives, and deriving a decomposition of system objectives into local objectives remains challenging for dynamic objectives such as temperature control. Although prior work has explored cooperative decentralized control or neural controllers independently, no existing approach has fully integrated cooperative decentralized control and (deep) neural networks to optimize resource control. So far, the literature has only presented trials that use multiple neural controllers for scalability and to manage complexity, but none of them has addressed resource control with a dedicated cooperative reinforcement learning algorithm. In this work, we propose a resource control framework based on a state-of-the-art cooperative reinforcement learning algorithm [21], following the principle of *centralized training for decentralized execution*.

B. Background: Cooperative Control with Reinforcement Learning

Reinforcement learning involves an agent interacting with an environment (observing the state of the environment s and taking actions a to influence it) in order to carry out a certain task, formulated as the maximization of a scalar feedback signal (reward r) [1]. Distributed resource control can be formulated as a cooperative reinforcement learning problem, where multiple agents, placed in a shared environment, need to collaborate to achieve a shared objective (maximize a shared reward signal). The objective could be, for example, to minimize the system's energy-delay product or maximize the average application throughput while maintaining power and thermal constraints. Typically, agents perceive a local subset o_i of the state of the underlying environment s [6]. The agents follow a policy $\pi(a|o_i)$ that maps these local observations to actions. Nevertheless, the environment is shared among all agents, so that both the shared reward and the environment's state (and in turn the local observations) are influenced by the joint behavior of all agents. Cen-

tralized training and decentralized execution is a popular paradigm to enable cooperation among agents [6]. Under this paradigm, agents act independently but collaborate for the optimization of their control policies. The actor-critic family of algorithms aligns naturally with this paradigm [6]. In centralized training and decentralized execution under the actor-critic paradigm, there are multiple actors (neural networks that learn the local policies) and a single centralized critic (a neural that learns to evaluate them). Each policy is conditioned on its local observation $\pi(a|o_i, \theta_i)$ and parameterized by a set of parameters θ_i . The central critic is trained as a regression model with a dedicated set of parameters ϕ and predicts the expected reward $Q(s, \mathbf{a}, \phi)$ under the joint behavior of the actors $\mathbf{a} = \{a_1 \dots a_n\}$ and conditioned on the underlying system state s . At a high level, the critic evaluates actions based on their influence on the achievable reward, while the actors adjust their policies to increase (decrease) the likelihood of selecting actions that yield high (low) rewards.

III. CENTRALIZED TRAINING AND DECENTRALIZED CONTROL FOR HIGHLY OPTIMIZED MULTICORES

We propose a cooperative resource control framework based on centralized training and decentralized execution following the actor-critic paradigm. Prior research has primarily focused on learning neural control policies, often paired with rule-based, task-specific resource allocation schemes. We argue that learning how to optimally assign resources to controllers is just as important as training the local controllers, as it opens the way for a more powerful control that supports complex, system-wide objectives, expressed as a shared reward signal, without in-depth prior knowledge about the processor and applications. To this end, we implement a state-of-the-art actor-critic algorithm [21] that uses neural networks to learn both the control policies (through the actor networks) and the resource allocation strategies (through the centralized critic network) at runtime. A major advantage of our approach is that the local controllers can execute in parallel without the need to communicate with other controllers or with a central arbiter to arrive at the final control decision. The control decisions are evaluated *in hindsight* (outside the latency critical control loop) during centralized training, when the controllers share their recent experiences (state observations and actions) with the critic and receive feedback about their control decision, allowing co-operative coordination of the system's resources. To illustrate how our approach works in practice, we implement a technique that follows the aforementioned scheme of centralized training and decentralized control to solve the problem of performance maximization under a temperature constraint using DVFS.

A. Problem Formulation

We focus on a processor with N_c cores with per-core clock signals and N_f discrete frequency levels. The goal is to find a frequency allocation across cores that maximizes the average throughput among concurrently running applications while making sure that the maximum core temperature remains below a critical threshold T_{crit} . We measure performance as the inverse of execution time: t_{exe}^{-1} . Multiple, multi-threaded applications execute in parallel and contend for the processor's thermal budget. We consider an open system, where the applications and their arrival times are considered unknown at design time. We assign incoming applications to the first free cores¹ and each core executes one thread of the application. The initial mapping remains fixed throughout the application's execution.

¹Our DVFS control is orthogonal to mapping decisions; it can be combined by different mapping policies.

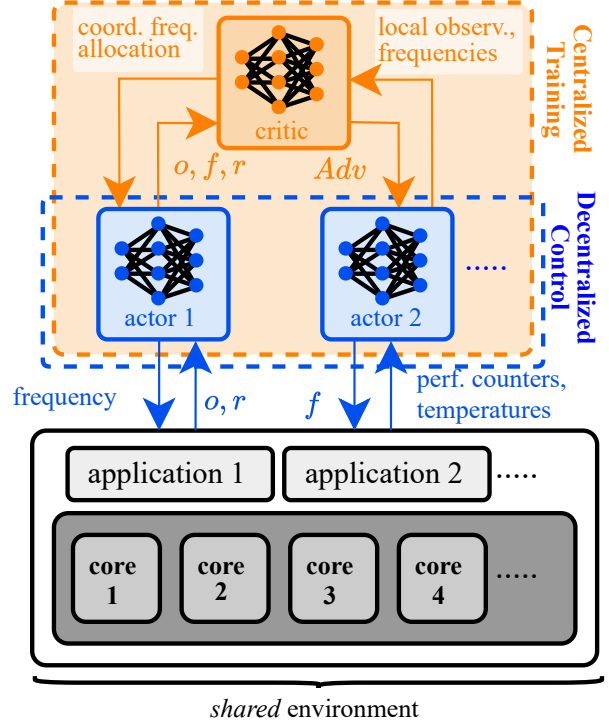


Fig. 2. Decentralized temperature control incorporating the actor-critic paradigm. Local actors implement the control policies, managing the core frequencies of an associated application. The actors engage in centralized training to coordinate their frequency decisions to maximize the average system performance under the processor's thermal budget. The coordination is implicitly performed by the central critic which evaluates the controllers' local frequency decisions within the system-wide context and with regard to the shared performance objective and temperature constraint. The actors align their decision to the frequency allocation recommended by the critic.

The problem can be summarized as follows.

objective: max. system perf. (avg. t_{exe}^{-1} across apps.)
constraint: $\max_c T_c \leq T_{crit}$
means: per-core DVFS

B. Our Approach using the Actor-Critic Paradigm

We propose using an actor-critic algorithm to solve this optimization problem. Each application is managed by a dedicated controller (actor) that adjusts the frequency of the cores associated with the application. We opt for agents at the application level, as they have knowledge about different threads of a multi-threaded application. Furthermore, it ensures that all threads of the application run at the same speed, preventing stalls at synchronization points [22]. To achieve the objective outlined above, we propose a shared reward signal that consolidates the throughputs of the different applications and enforces a system-wide temperature constraint. Due to the spatial and temporal heat transfer, system-wide thermal control cannot be easily decomposed into local, independent sub-problems for each controller. Such a decomposition would require detailed information about the processor's thermal behavior (e.g., floorplan, device node) and cooling system. In our approach, the *central critic* evaluates the frequency allocation considering the interactions of all local DVFS controllers with the processor. Using the representational power of neural networks, the critic learns the complex interdependencies between core activity, performance, and thermal behavior. An overview of our approach is provided in Fig. 2. It is described in detail in the next sections.

1) *State, Reward, and Actions*: We first focus on the local controllers and formalize the state and action spaces, as well as the shared reward signal.

State (Observation): Each controller can consider only a fraction of the underlying processor state to derive its control decisions. We use the following features in the observation of each controller. The frequency, average instructions per cycle (IPC), last-level cache misses, and utilization across the cores under the controller's supervision during the previous execution interval, as well as their current average and maximum temperature. These features characterize the performance and thermal behavior of the application. The impact of a frequency change is highly application-specific. Furthermore, different execution phases of the same application require different frequency selection. In general, compute-intensive application phases benefit from higher frequencies. During memory-intensive phases (characterized by many misses on the last-level cache), computation happens faster than new data arrive at the processor, and consequently, increasing the frequency will not convert into performance gains. At the same time, a higher frequency always leads to increased dynamic power consumption ($P_{dynamic} \sim f^3$) and consequently higher temperature. Therefore, considering these application characteristics is crucial for an adaptive frequency selection that balances potential performance gains and increasing temperature.

(Shared) Reward: Temperature control requires balancing potential performance gains between multiple applications while maintaining thermally-safe operation. We propose a shared reward signal that consolidates these two objectives. The first part of the reward is based on a shared performance metric, which uses the geometric mean of the applications' instructions per second (IPS) as a surrogate for overall system performance Eq. (1).

$$r_t = \begin{cases} \text{violation-penalty} & \exists c \in \{1 \dots N_c\} \text{ s.t. } T_{t+1}^c > T_{crit} \\ \left(\prod_{i=1}^{N_a} ips_{t+1}^i \right)^{\frac{1}{N_a}} & \text{otherwise} \end{cases} \quad (1)$$

Here, N_a refers to the number of applications currently running. This metric encourages agents to adjust their frequencies in a way that improves the collective performance, without favoring any particular application. It also promotes fairness by tending toward a more balanced performance distribution across applications: The geometric mean increases as the performance deviation among the applications decreases. The second part of the reward penalizes all agents when the temperature constraint is exceeded on any core of the system. This is crucial, as thermal violations can result from frequency increases on other cores (by other controllers) and/or at earlier points in time, due to spatial and temporal thermal effects. By penalizing all agents, the reward emphasizes the entangled nature of the problem.

Actions: The controllers select from the entire set of discrete frequency levels exposed by the processor, thereby allowing fine-grained and opportunistic DVFS. Each frequency has an associated voltage level that is automatically set when the frequency is changed.

2) *Decentralized Control*: During the decentralized execution phase, the actors are periodically invoked within the control loop (see Algorithm 1) and execute in parallel relying only on their local observations. The number of active controllers depends on the utilization of the system. The controllers log their experiences into a local buffer, maintaining a trajectory of the most recent interactions $(o_{i,t}, f_{i,t})$. In practice, the reward is not directly emitted by the environment. It suffices to *compute* reward in hindsight during the centralized training phase, combining the observations of all agents. Therefore, the controllers also store the IPS and the maximum temperature among their cores. When the trajectory reaches a predefined length, one round of centralized training is initiated.

Algorithm 1 Decentralized Temperature Control

```

for each actor  $i$  do ▷ in parallel
  while local buffer  $\leq$  max-traj-length do
    acquire local state  $o_i$ 
    sample next frequency  $f_i \sim \pi(f_i|o_i, \theta_i)$ 
    set core frequency  $f_i$ 
    ... ▷ application process
    monitor  $ips_i, T_i^{max}$ 
    store  $(o_i, f_i, ips_i, T_i^{max})$  in local buffer
  optimize critic  $\phi$  and local actors  $\{\theta_i\}$ 
  repeat with updated actors

```

3) *Centralized Training*: Centralized training involves training the centralized critic and then updating control policies [21]. The optimization of the critic is described in Algorithm 2. First, the shared reward is calculated from the local IPS and peak temperature measurements according to Eq. (1). This is done for every time step within the trajectory. Afterward, the state of the central critic is composed. The critic considers the local trajectories of all controllers and their selected actions [21]. We compute a central state for each controller which is the concatenation of its own observation o_i and the mean of the observations of the concurrently running controllers o_{-i} . Furthermore, we add the maximum f_{-i}^{max} and mean f_{-i}^{avg} frequency selections of the concurrently running controllers and the number of active applications (controllers) N_a .

$$s_i = o_i \cup o_{-i} \cup \{f_{-i}^{max}, f_{-i}^{avg}, N_a\} \quad (2)$$

The critic state is computed for every controller and for every timestep in the trajectory. It is fed through the critic network to provide an estimate of the expected reward (called value) for each of the agent's actions given its own observation o_i plus the system-wide context $o_{-i} \cup \{f_{-i}^{max}, f_{-i}^{avg}, N_a\}$. Since all value predictions are derived with the same network ϕ , they can be computed in a batch over the length of the trajectory and the number of actors. The $TD(\lambda)$ algorithm [1] and a delayed target network ϕ to recursively compute regression targets, labeled Y^γ , for the optimization of the critic network.

$$Y_{i,t}^\lambda = r_t + \gamma \cdot \left((1 - \lambda) \cdot Q(s_{i,t+1}, f_{i,t+1}, \phi') + \lambda \cdot Y_{i,t+1}^\lambda \right) \quad (3)$$

The target value at the final time step of the trajectory is set to $Y_T^\gamma = Q(s_{i,T}, f_{i,T}, \phi')$. We show the temporal index here because it is relevant for recursive computation. We implicitly assume that the steps outlined in the algorithms need to be executed for all time steps, and therefore, for reasons of readability, do not show the temporal index there. The critic's parameters are updated using gradient descent on the mean-squared error loss, completing the optimization of the critic Eq. (4).

$$\mathcal{L}_Q = (Q(s, f, \phi) - Y^\lambda)^2 \quad (4)$$

The loss is computed over across all timesteps in the trajectory and all actors. This completes the optimization of the central critic.

The reward estimates of the critic are used to derive the feedback for each controller's frequency selection, which is optimized in the second step of centralized training (see Algorithm 3). This feedback is called the action's advantage. It is the difference between the critic's value prediction and an action-independent baseline. The purpose of this subtraction is to reduce the variance of the subsequent update of the control policies [1]. To compute the advantage term, we build on the counterfactual baseline introduced in [21] (see Eq. (5)).

$$Adv(s_i, f_i, \phi) = Q(f_i, s_i, \phi) - \sum_{j=1}^{N_f} \pi(f_j, o_i, \theta) \cdot Q(f_j, s_i, \phi) \quad (5)$$

Algorithm 2 Centralized Training (Critic Network)

```

for each time step in traj. do                                ▷ batch processing
  compute reward  $r$  from local  $ips_i, T_i^{max}$                     ▷ Eq. (1)
  for each actor  $i$  do                                          ▷ batch processing
    compute critic states  $s_i$                                     ▷ Eq. (2)
    compute critic values  $Q(s_i, f_i, \phi)$ 
    compute target critic values  $Q(s_i, f_i, \phi')$ 
  for each time step in traj do                                ▷ sequential processing
    for each actor  $i$  do                                          ▷ batch processing
      compute  $TD(\lambda)$  targets  $Y^\lambda$                         ▷ Eq. (3)
    compute critic loss  $\mathcal{L}_Q$                                 ▷ Eq. (4), (mean over traj. & actors)
    update critic params.  $\phi \leftarrow \phi + \alpha \cdot \frac{\partial \mathcal{L}_Q}{\partial \phi}$ 
    update target critic params.  $\phi' \leftarrow \tau \cdot \phi + (1 - \tau) \cdot \phi'$ 

```

Algorithm 3 Centralized Training (Actor Networks)

```

for each step in traj. do                                    ▷ batch processing
  for each actor  $i$  do                                          ▷ batch processing
    compute critic values  $Q(s_i, f_i)$ 
  for each actor  $i$  do                                          ▷ in parallel
    for each step in traj do                                    ▷ batch processing
      compute freq. distr.  $\pi(f|o_i, \theta_i)$ 
      compute advs.  $Adv(o_i, f_i)$                             ▷ Eq. (5)
    compute  $\mathcal{L}_{\pi,i}$                                           ▷ Eq. (6), mean over traj.
    update actor params.  $\theta_i \leftarrow \theta_i - \alpha_\pi \cdot \frac{\partial \mathcal{L}_{\pi,i}}{\partial \theta_i}$ 

```

The counterfactual advantage measures the difference of the return of the selected frequency and the average return assuming that all other controllers keep their current frequency selection $\{f_{-i}^{max}, f_{-i}^{avg}\}$. It answers the question: "Given the concurrent workload, how does a controller's frequency decision compare to all other possible frequencies?" therefore facilitating credit assignment (see Fig. 1).

To optimize the control policies $\pi(a|o_i, \phi)$, the advantage term is used as part of the loss function (see Eq. (6)), guiding the actors toward frequencies with high predicted reward while avoiding those with low predicted reward.

$$\mathcal{L}_\pi = -\log(\pi(f_i, o_i, \theta_i)) \cdot Adv(s_i, f_i, \phi) - c_H \cdot H(\pi(f|o_i, \theta_i)) \quad (6)$$

Here, $\pi(f_i, o_i, \theta_i)$ is the probability of the frequency that was actually chosen by controller i , while $\pi(f_i|o_i, \theta_i)$ is the probability distribution conditioned on the observation o_i . The entropy term $c_H \cdot H(\pi(f|o_i, \theta))$ with parameter c_H is used to balance exploration and exploitation [23], [24]. The loss is averaged over the entire trajectory.

4) *Configuration & Parameters*: The local controllers are invoked every 10 ms. Both the actor and the critic network use two layers with 64 neurons. We use the *tanh* non-linearity. The learning rate of the critic (0.001) is set one order of magnitude higher than the one of the actors (0.0001) [25]. The maximum trajectory length is set to 128, resulting in one round of centralized training every 1.3 s. We set the violation penalty to -100 and scale the reward to be in the range of -1 and 1 [26]. We use a discount factor of 0.9. For the target update rate τ and for λ , we follow the recommendations from the literature [21], [25]. We set the entropy coefficient to 0.001 and normalize the state vectors of the actor and critic using running estimates of the mean and variance for each state feature [26]. In addition, we use weight sharing between actors, i.e., $\theta = \theta_1 = \dots = \theta_n$ [21].

IV. EVALUATION

To evaluate our framework, we implement the actor-critic architecture on a real Intel® Core™ i9-12900KF processor [27], which features eight performance cores with individual clock lines and 44 frequency levels ranging from 0.8 GHz to 5.1 GHz. We implement

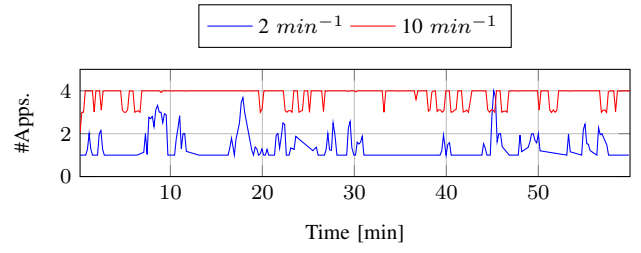


Fig. 3. Number of concurrently running applications demonstrating the varying degrees of system utilization to which the actors and the critic need to adapt. Higher utilization leads to more contention for the thermal budget, while low utilization allows individual applications to opportunistically maximize their performance. Data recorded for two and ten applications per minute under our technique.

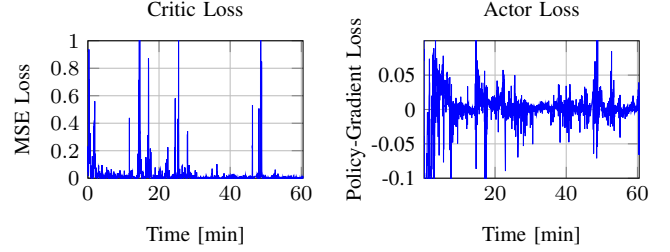


Fig. 4. Loss curves for the critic and actor networks during the first hour of the evaluation. The critic learns faster since it needs to estimate the actors' current behavior [31]. Short spikes in the critic loss are likely caused by previously unseen workload combinations. The loss quickly goes back to a low value indicating the critic's capability to adapt. Data recorded for the arrival rate of six applications per minute.

a Linux kernel module to read the performance counters and set the core frequencies. A userspace program implements the actor and critic algorithms and uses *libtorch* [28] for neural network training and inference.

A. Evaluation Workloads and Scenarios

We evaluate our approach using the following applications from the benchmark suites *PARSEC* [29] and *SPLASH-2* [30]: *barnes*, *fft*, *fluidanimate*, *lu*, *lu* (n.c.), *ocean*, *ocean* (n.c.), *swaptions*, *radiosity*, and *radix*. This workload constitutes a mixture of diverse application characteristics, with varying execution phases, peak performance, and maximum temperature. To test the techniques under varying system utilization levels (and consequently varying degrees of contention for the thermal budget), we conduct evaluation runs with five different application arrival rates: two, four, six, eight, and ten arriving applications per minute. To maximize workload diversity across the eight cores, each application uses two threads. This setup results in frequent changes to the optimal frequency allocation between applications, requiring the controllers to adapt their frequency selection dynamically (see Fig. 3). For all experiments, we use a temperature constraint T_{crit} of 75°C. To fairly compare the approaches, we run them for two hours, giving them enough time to learn the characteristics of the environment. We report the evaluation results for the first hour separately from the second hour. During the first hour, the controllers explore various frequency selections, and the critic learns which frequency allocations are beneficial for thermally-safe and performant operation. Fig. 4 shows the loss curve for our critic and actor networks during this phase, exemplarily for a scenario with six applications per minute. In the second hour, the controllers apply the learned policies while still being able to adapt to unseen workload combinations.

B. Comparison Baselines

1) *TP-DQL* [10]: It uses one neural controller per core to maximize application performance under a thermal constraint. It is the

closest technique to our work, as it targets the same problem and uses neural controllers. The controllers share the same neural network and use deep Q-learning with a shared replay buffer, to enable cooperation through knowledge sharing. The reward signal is individual to each core, and each agent tries to maximize the core frequency while keeping the core temperature just below the threshold.

2) *Profit* [9]: It is based on tabular Q-learning. It is designed for power-constrained performance optimization and uses per-core controllers that each train an individual policy. A central coordinator distributes local power budgets to each local controller according to the cores' IPC. This heuristic for deriving local constraints is not applicable for temperature control. Therefore, we change the local power constraint to a fixed temperature constraint. We use the same penalty value as in our reward function.

C. Evaluation Results

Figure 5 shows the results of our evaluation, in terms of the execution time of applications, the IPS, and also the number of thermal violations. Our method increases overall performance across applications by 20% and 34% (average/maximum) compared to *TP-DQL*, and by 24% and 65% (average/maximum) when compared to *Profit*. As the application arrival rate increases, so does the average execution time, due to increased contention for the thermal budget. In such cases, controllers must resort to lower frequencies to keep the temperature within the critical threshold. Importantly, the room for performance optimization is significantly reduced. That explains why all comparison techniques show similar performance at the arrival rate of ten applications per minute, at which the utilization of the processor reaches 100%. All techniques manage to keep the average temperature below the critical threshold, with our approach closing the thermal margin to the threshold, effectively using the chip's thermal budget. Looking at the thermal violation results, one can observe that all comparison techniques manage to avoid thermal violations in the exploitation phase. For our technique, the maximum violation during the exploitation phase is equal to 0.05 violations per control epoch, which is minimal. However, during the exploration phase, where the agents are supposed to select random actions (frequency selections) to learn the consequence of these actions, we notice that the agents have tested high frequencies, resulting in some violations (0.2 violations per control epoch) in the scenario where system utilization reaches 100%. Experiencing such scenarios lets the actors learn to avoid high frequencies at high system utilization to not violate the temperature constraint. Therefore, we can directly see that in the exploitation phase, our approach experiences almost no violations, demonstrating the ability to adapt to the high system utilization.

The improvement in overall system performance, both with regard to response time and throughput, while maintaining a comparable rate of thermal violations, highlights the superiority of our approach over the state of the art.

D. Overhead Discussion

Our resource control runs on the same cores as the applications. The main contributors to the overhead of our approach is the inference overhead as part of the control loop and the optimization of the actor and the critic networks during centralized training. Inference of the actor network(s) and sampling the next frequency on average takes 565 μ s in each control epoch, i.e., 10ms, in our evaluation. Centralized training occurs every 1.3 s and takes 20 ms on average. In summary, the inference overhead is 5.7 % and the optimization overhead is 1.6 %. The overhead of the entire control loop, including reading the performance counters and associated context switches, as well as the overhead of the optimization, is already considered in our

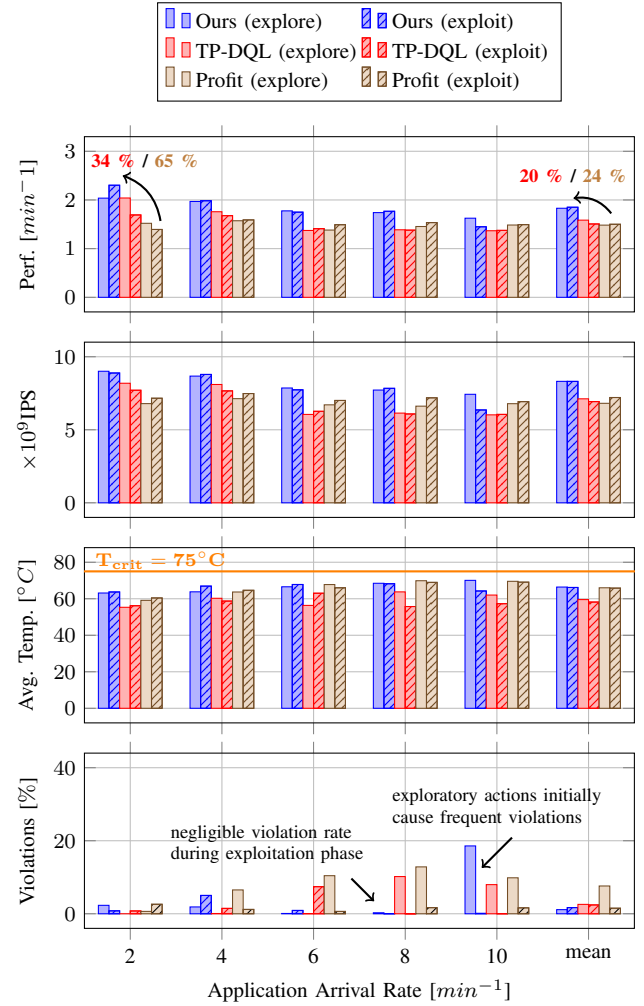


Fig. 5. Results of our evaluation with regard to performance (t_{exe}^{-1}), throughput, temperature and the percentage of thermal violations throughout the execution.

evaluation results, since we compute the performance improvements using the execution time which comprises the application runtime and the overhead of the techniques. Our technique stands out with a light memory footprint, as it only needs to store the most recent experiences for each controller. Compared to *TP-DQL*, which relies on experience replay and stores 100,000 samples, the memory overhead of our technique is 99 % lower. The total size of all buffers never exceeds 20 kB.

V. CONCLUSION

Deep reinforcement learning is a promising approach to processor resource control. Deployment to real hardware requires multiple controllers to handle the vast solution space involved. However, distributed decision-making introduces additional challenges, such as interference between concurrent controllers and partial observability, that have not been sufficiently addressed by prior resource control techniques. This work addressed these challenges using a state-of-the-art actor-critic algorithm and achieves cooperative temperature control while maintaining the scalability of distributed decision-making. We have solved an outstanding problem, and we report significant performance gains that represent from now on the state of the art for the problem of thermal-aware multicore computing.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [2] Y. Wang, W. Zhang, M. Hao, and Z. Wang, "Online power management for multi-cores: A reinforcement learning based approach," *IEEE Trans. Parallel Distributed Syst.*, vol. 33, no. 4, pp. 751–764, 2022.
- [3] J. Zhang, G. Yu, Z. He, L. Ai, and P. Chen, "Deeppower: Deep reinforcement learning based power management for latency critical applications in multi-core systems," in *52nd International Conference on Parallel Processing, ICPP*. ACM, 2023, pp. 327–336.
- [4] S. Yang, Y. Wang, D. Liu, X. Jiang, H. Fang, Y. Yang, and M. Zhao, "Releta: Reinforcement learning for thermal-aware task allocation on multicore," *CoRR*, vol. abs/1912.00189, 2019.
- [5] D. Liu, S. Yang, Z. He, M. Zhao, and W. Liu, "CARTAD: compiler-assisted reinforcement learning for thermal-aware task scheduling and DVFS on multicore," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, vol. 41, no. 6, pp. 1813–1826, 2022.
- [6] A. Wong, T. Bäck, A. V. Kononova, and A. Plaata, "Deep multiagent reinforcement learning: challenges and directions," *Artif. Intell. Rev.*, vol. 56, no. 6, pp. 5023–5056, 2023.
- [7] R. Bitirgen, E. Ipek, and J. F. Martínez, "Coordinated management of multiple interacting resources in chip multiprocessors: A machine learning approach," in *41st Annual IEEE/ACM International Symposium on Microarchitecture MICRO*. IEEE Computer Society, 2008, pp. 318–329.
- [8] G. Mariani, G. Palermo, C. Silvano, and V. Zaccaria, "ARTE: an application-specific run-time management framework for multi-core systems," in *IEEE 9th Symposium on Application Specific Processors, SASP*. IEEE Computer Society, 2011, pp. 86–93.
- [9] Z. Chen, D. Stamoulis, and D. Marculescu, "Profit: Priority and power/performance optimization for many-core systems," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, vol. 37, no. 10, pp. 2064–2075, 2018.
- [10] H. Khdr, M. E. Batur, K. Zhou, M. B. Sikal, and J. Henkel, "Multi-agent reinforcement learning for thermally-restricted performance optimization on manycores," in *Design, Automation & Test in Europe Conference & Exhibition, DATE*. IEEE, 2024, pp. 1–6.
- [11] D. Huang, L. Costero, and D. Atienza, "Is the powersave governor really saving power?" in *24th International Symposium on Cluster, Cloud and Internet Computing, CCGrid*. IEEE, 2024, pp. 273–283.
- [12] S. Kim, K. Bin, S. Ha, K. Lee, and S. Chong, "ztt: learning-based DVFS with zero thermal throttling for mobile devices," in *19th Annual International Conference on Mobile Systems, Applications, and Services, MobiSys*. ACM, 2021, pp. 41–53.
- [13] A. Zou, Y. Ma, K. Garimella, B. Lee, C. D. Gill, and X. Zhang, "F-LEMMA: fast learning-based energy management for multi-/many-core processors," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, vol. 42, no. 2, pp. 616–629, 2023.
- [14] Q. Zhou, Y. Zhang, X. Xu, Q. Zhang, H. Wu, Y. Lian, and Y. Zhao, "A lightweight drdp-based RL DVFS for video rendering on CPU-GPU integrated soc," *IEEE Trans. Circuits Syst. I Regul. Pap.*, vol. 71, no. 5, pp. 2119–2131, 2024.
- [15] J. Li, W. Jiang, Y. He, Q. Yang, A. Gao, Y. Ha, E. Ozcan, R. Bai, T. Cui, and H. Yu, "FiDRL: Flexible Invocation-Based Deep Reinforcement Learning for DVFS Scheduling in Embedded Systems," *IEEE Transactions on Computers*, no. 01, pp. 1–14, Sep. 5555.
- [16] Y. Bai, V. W. Lee, and E. Ipek, "Voltage regulator efficiency aware power management," in *22nd International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS*. ACM, 2017, pp. 825–838.
- [17] B. Donyanavard, T. Mück, A. M. Rahmani, N. D. Dutt, A. Sadighi, F. Maurer, and A. Herkersdorf, "SOSA: self-optimizing learning with self-adaptive control for hierarchical system-on-chip management," in *52nd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO*. ACM, 2019, pp. 685–698.
- [18] R. Jain, P. R. Panda, and S. Subramoney, "Cooperative multi-agent reinforcement learning-based co-optimization of cores, caches, and on-chip network," *ACM Trans. Archit. Code Optim.*, vol. 14, no. 4, pp. 32:1–32:25, 2017.
- [19] L. Costero, A. Iranfar, M. Zapater, F. D. Igual, K. Olcoz, and D. Atienza, "Resource management for power-constrained HEVC transcoding using reinforcement learning," *IEEE Trans. Parallel Distributed Syst.*, vol. 31, no. 12, pp. 2834–2850, 2020.
- [20] G. Pan, J. Jou, and B. Lai, "Scalable power management using multilevel reinforcement learning for multiprocessors," *ACM Trans. Design Autom. Electr. Syst.*, vol. 19, no. 4, pp. 33:1–33:23, 2014.
- [21] J. N. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *32nd Conference on Artificial Intelligence, AAAI*. AAAI Press, 2018, pp. 2974–2982.
- [22] A. R. Alameldeen and D. A. Wood, "IPC considered harmful for multiprocessor workloads," *IEEE Micro*, vol. 26, no. 4, pp. 8–17, 2006.
- [23] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Mach. Learn.*, vol. 8, pp. 229–256, 1992.
- [24] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *33rd International Conference on Machine Learning, ICML*, ser. JMLR Workshop and Conference Proceedings, vol. 48. JMLR.org, 2016, pp. 1928–1937.
- [25] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *4th International Conference on Learning Representations, ICLR*, 2016.
- [26] M. Andrychowicz, A. Raichuk, P. Stanczyk, M. Orsini, S. Girgin, R. Marinier, L. Hussenot, M. Geist, O. Pietquin, M. Michalski, S. Gelly, and O. Bachem, "What matters in on-policy reinforcement learning? A large-scale empirical study," *CoRR*, vol. abs/2006.05990, 2020.
- [27] "Intel® core™ i9-12900kf processor," <https://www.intel.com/content/www/us/en/products/sku/134600/intel-core-i912900kf-processor-30m-cache-up-to-5-20-ghz/specifications.html>, Intel Corporation, 2021, (accessed: 2024-11-15).
- [28] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Z. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *32th Annual Conference on Neural Information Processing Systems, NeurIPS*, 2019, pp. 8024–8035.
- [29] C. Bienia, S. Kumar, J. P. Singh, and K. Li, "The PARSEC benchmark suite: characterization and architectural implications," in *17th International Conference on Parallel Architectures and Compilation Techniques, PACT*. ACM, 2008, pp. 72–81.
- [30] S. C. Woo, M. Ohara, E. Torrie, J. P. Singh, and A. Gupta, "The SPLASH-2 programs: Characterization and methodological considerations," in *22nd International Symposium on Computer Architecture, ISCA*. ACM, 1995, pp. 24–36.
- [31] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *35th International Conference on Machine Learning, ICML*, ser. Proceedings of Machine Learning Research, vol. 80. PMLR, 2018, pp. 1582–1591.